Lab Report No 4
Implementation of Gray Level Transformation
Digital Image Processing

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

24th Oct,2024

Department of Computer Engineering,

HITEC University, Taxila

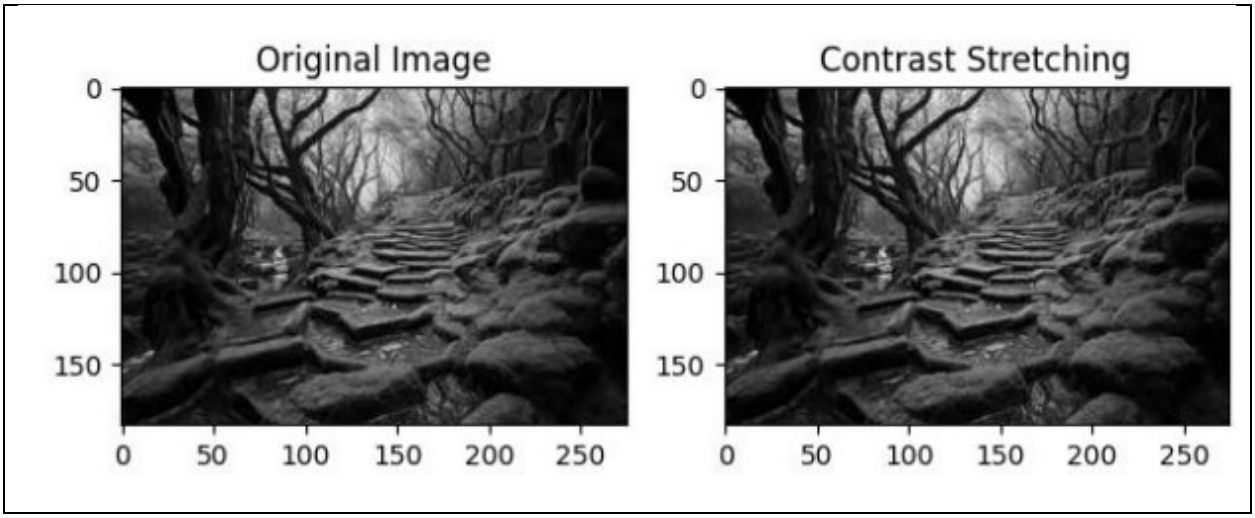
Lab Example No 1:

Solution:

Brief description (3-5 lines)
Contrast stretching is a simple image enhancement technique that attempts to improve the contrast in an image by `stretching' the range of intensity values it contains to span a desired range of values. Normally (min, max) → (0, 255)
The code

import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image in grayscale 	
image = cv2.imread('Screenshot 2024-10-26 114803.jpeg', 0)
r_min, r_max = np.min(image), np.max(image)
contrast_stretched = (image - r_min) * (255 / (r_max - r_min))
# Normalize to [0, 255] 	
contrast_stretched = np.array(contrast_stretched, dtype=np.uint8)
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')
# Display the result 	
plt.subplot(1, 2, 2)
plt.imshow(contrast_stretched, cmap='gray')
plt.title('Contrast Stretching')
plt.tight_layout()
plt.show()

The results (Screenshot)



Lab Example No 2:

Solution:

Brief description (3-5 lines)
Thresholding is the operation through which an image can be converted into a binary image/black & white i.e. having only two distinct levels. Threshold value can be the mean or median etc. of the image. Each pixel is compared to a threshold value: • Pixels greater than the threshold are set to one value (e.g., 255). • Pixels below the threshold are set to another value (e.g., 0).
The code

import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image in grayscale 	
image = cv2.imread('Screenshot 2024-10-26 122352.jpeg', 0)
# Apply simple thresholding 	
# Threshold at 127, values above 127 are set to 255, below are set to 0 	
_, binary_image = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)
# Apply inverse binary thresholding 	
_, binary_inv_image = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY_INV)

```
# Display original and thresholded images
plt.figure(figsize=(10, 5))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(binary_image, cmap='gray')
plt.title('Binary Thresholding')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(binary_inv_image, cmap='gray')
plt.title('Inverse Binary Thresholding')
plt.axis('off')
plt.tight_layout()
plt.show()
```

The results (Screenshot)

Lab Example No 3:

Solution:

Brief description (3-5 lines)
Gray level Slicing is used to highlight a specific range of gray levels in an image. It can either: Highlight a specific intensity range while keeping other intensities intact. Highlight a specific intensity range while suppressing all other intensities to black (0).
The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image in grayscale
image = cv2.imread('Screenshot 2024-10-26 122930.png',
cv2.IMREAD_GRAYSCALE)
# Define the gray-level slicing function
def gray_level_slicing(image, min_val, max_val, slice_value):
    # Create an output image filled with zeros (black)
    output_image = np.zeros_like(image)
    # Set the pixel values within the specified range to the slice_value
    output_image[(image >= min_val) & (image <= max_val)] =
slice_value
    return output_image
# Apply gray-level slicing to highlight pixels in the range [100, 200]
sliced_image = gray_level_slicing(image, 100, 200, 255)
# Display original and gray-level sliced images
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(sliced_image, cmap='gray')
plt.title('Gray Level Slicing')
plt.axis('off')
plt.tight_layout()
plt.show()
```

The results (Screenshot)

Lab Example No 4:

Solution:

Brief description (3-5 lines)
Transformation operations help enhance the quality of the image by applying operations like log, inverse log and power on the entire image. Different type of transformation yields different results. **1. Power-Law (Gamma) Transformation** Gamma correction is used for adjusting brightness. The formula of power or gamma transformation is: $S=c.r^{y}$ where c and gamma are constant.
The code

import cv2
import numpy as np
import matplotlib.pyplot as plt
def gamma_correction(image, gamma):
# Normalize the image to range [0, 1] 	
normalized_image = image / 255.0
# Apply the gamma correction 	
gamma_corrected = np.power(normalized_image, gamma)
# Scale back to [0, 255] 	
gamma_corrected = np.uint8(gamma_corrected * 255)
return gamma_corrected

```
# Load the image in grayscale
image = cv2.imread('download.jpeg', 0)
# Define different gamma values for experimentation
gamma_values = [0.5, 1.0, 1.5, 2.0]
# Plot original and gamma corrected images
plt.figure(figsize=(12, 8))
# Display original image
plt.subplot(2, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
# Apply Gamma Transformations for each gamma value
for i, gamma in enumerate(gamma_values):
gamma_image = gamma_correction(image, gamma)
# Display the gamma corrected image
plt.subplot(2, 3, i + 2)
plt.imshow(gamma_image, cmap='gray')
plt.title(f'Gamma = {gamma}')
plt.axis('off')
plt.tight_layout()
plt.show()
```

The results (Screenshot)

Lab Example No 5:

Solution:

Brief description (3-5 lines)

Log transformation enhances darker pixels in an image. The formula of log transformation is:

s=c*log(1+r)

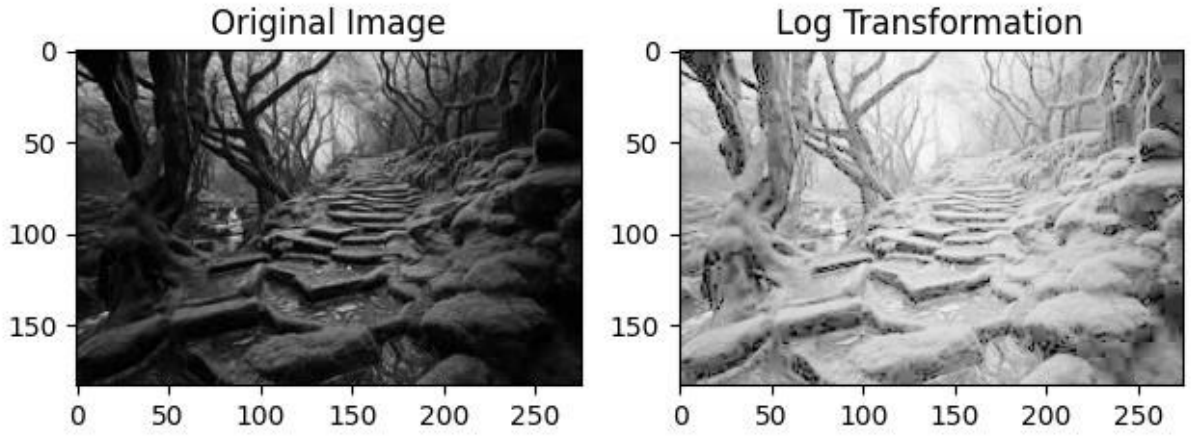
where c is a scaling constant and r is the pixel intensity.

c = 255 / log(1 + rmax)

The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image in grayscale
image = cv2.imread('download.jpeg', 0)
c = 255 / np.log(1 + np.max(image))
log_transformed = c * (np.log(1 + image))
# Normalize to [0, 255]
log_transformed = np.array(log_transformed, dtype=np.uint8)
# Display the result
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')
plt.subplot(1, 2, 2)
plt.imshow(log_transformed, cmap='gray')
plt.title('Log Transformation')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



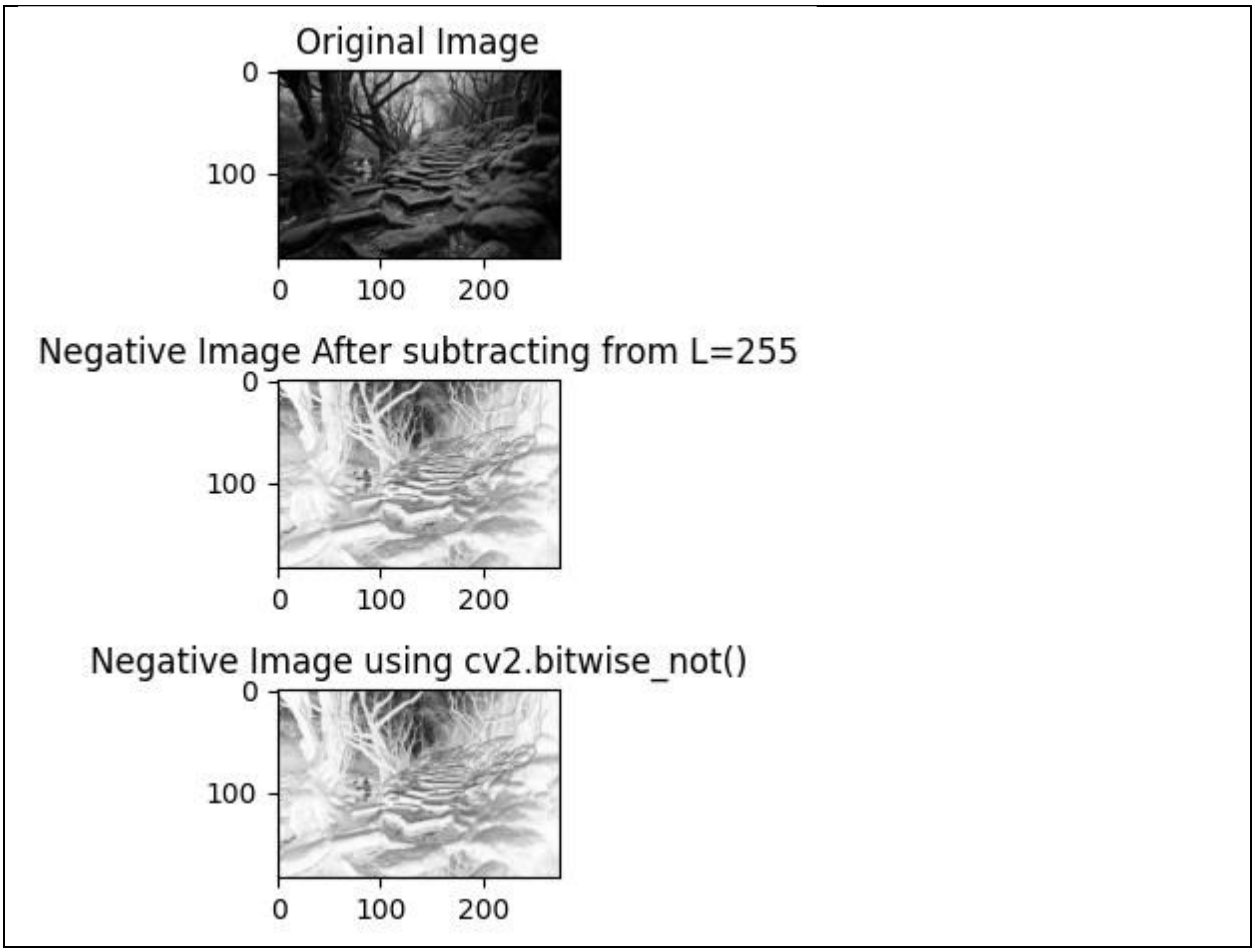
Lab Example No 6:

Solution:

Brief description (3-5 lines)
Image Negative: This technique inverts the intensity of pixels. The formula for inverse transformation is: **s=(L-1)-r** where L is the maximum pixel value, typically 256 for 8-bit images.
The code

import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image in grayscale 	
image = cv2.imread('download.jpeg', 0)
# Perform negative transformation 	
negative_image1 = 255 - image
# Perform negative operation using cv2.bitwise_not() 	
negative_image2 = cv2.bitwise_not(image)
# Display original and negative image 	
plt.subplot(3, 1, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')
plt.subplot(3, 1, 2)
plt.title('Negative Image After subtracting from L=255')
plt.imshow(negative_image1, cmap='gray')
plt.subplot(3, 1, 3)
plt.title('Negative Image using cv2.bitwise_not()')
plt.imshow(negative_image2, cmap='gray')
plt.tight_layout()
plt.show()

The results (Screenshot)



Lab Task No 1:

Solution:

Brief description (3-5 lines)
Apply contrast stretching on the image provided in lab by setting 5th and 95th percentiles of the input values to 0 and 255 respectively.
The code

import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image 	
img = cv2.imread('Screenshot 2024-10-26 122930.jpg', 0)

```
# Calculate the 5th and 95th percentiles
p5 = np.percentile(img, 5)
p95 = np.percentile(img, 95)
# Apply contrast stretching
stretched_img = np.clip((img - p5) * (255 / (p95 - p5)), 0,
255).astype(np.uint8)
# Display the original and stretched images
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(stretched_img, cmap='gray')
plt.title('Contrast Stretched Image')
plt.axis('off')
plt.tight_layout()
plt.show()
```

The results (Screenshot)

Lab Task No 2:

Solution:

Brief description (3-5 lines)

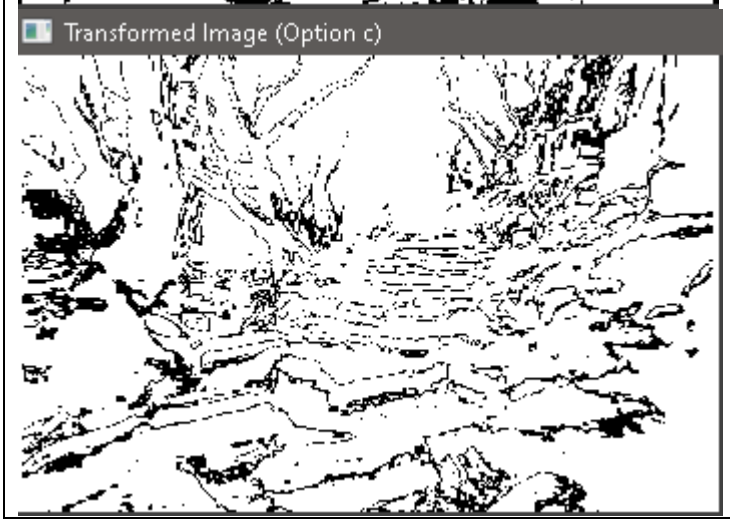
Apply the following transformation on an contrast enhanced image.

	Pixel values range	Output pixel value
a)	**Less than mean** **Greater than mean**	**0** **255**
b)	**Less than mean** **Greater than mean**	**255** **0**
c)	**±20 mean** **Otherwise**	**0** **255**

The code

```
import cv2
import numpy as np
# Load the image
image = cv2.imread('Screenshot 2024-10-26 200104.jpg',
cv2.IMREAD_GRAYSCALE)
# Enhance contrast
enhanced_image = cv2.equalizeHist(image)
# Calculate the mean pixel value
mean = np.mean(enhanced_image)
# Apply the transformation
# Option a)
transformed_image_a = np.where(enhanced_image < mean, 0,
255).astype(np.uint8)
# Option b)
transformed_image_b = np.where(enhanced_image < mean, 255,
0).astype(np.uint8)
# Option c)
transformed_image_c = np.where(np.abs(enhanced_image - mean) <=
20, 0, 255).astype(np.uint8)
# Display the results
cv2.imshow('Original Image', image)
cv2.imshow('Enhanced Image', enhanced_image)
cv2.imshow('Transformed Image (Option a)', transformed_image_a)
cv2.imshow('Transformed Image (Option b)', transformed_image_b)
cv2.imshow('Transformed Image (Option c)', transformed_image_c)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

The results (Screenshot)



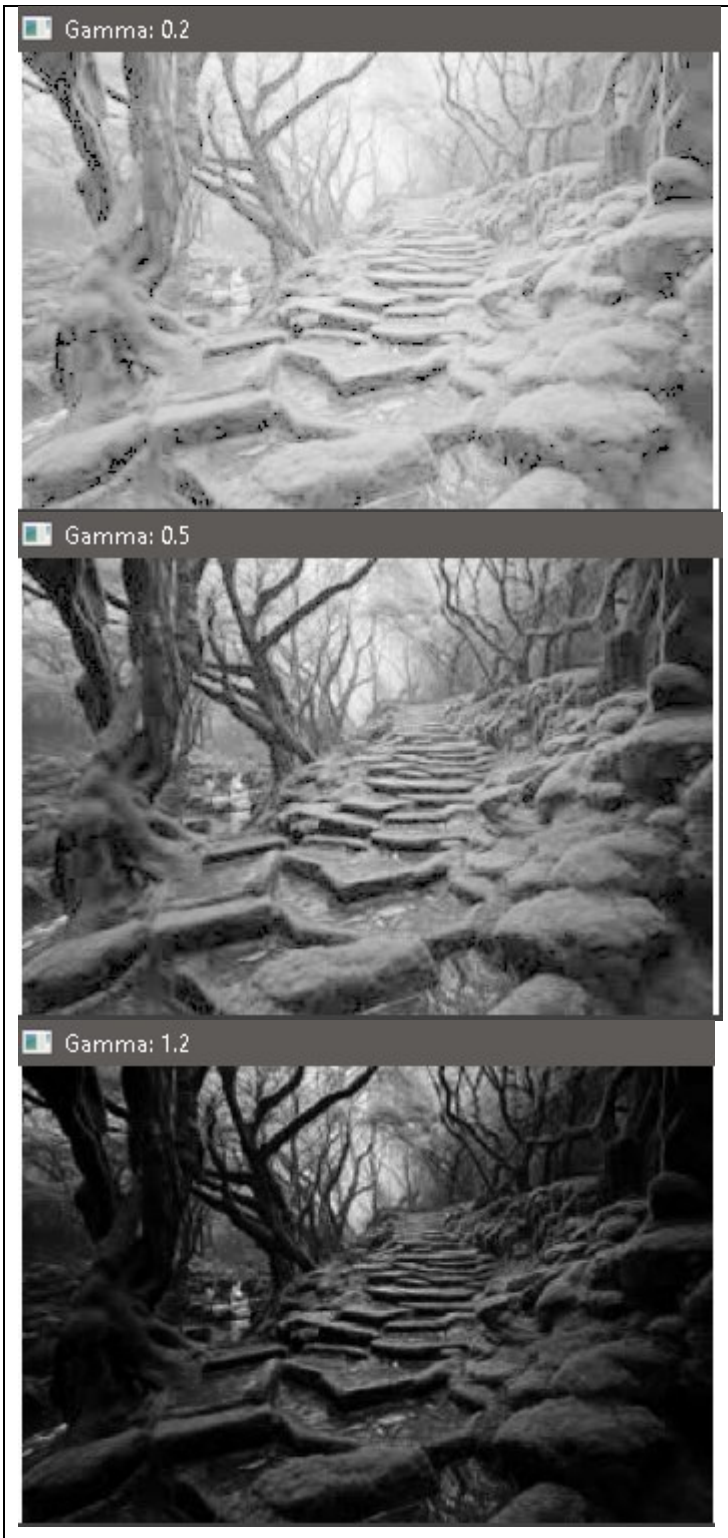
Lab Task No 3:

Solution:

Brief description (3-5 lines)
After obtaining constrast streatched images from task 1 . Apply Power Law transformation for the following values of γ (0.2, 0.5, 1.2 and 1.8) . Make sure to adjust data types accordingly
The code
import cv2 import numpy as np # Load the image

```
img = cv2.imread('Screenshot 2024-10-26 200104.jpg',
cv2.IMREAD_GRAYSCALE)
# Define the gamma values
gamma_values = [0.2, 0.5, 1.2, 1.8]
# Apply Power Law transformation for each gamma value
for gamma in gamma_values:
    # Convert image to float type
    img_float = img.astype(np.float32) / 255.0
    # Apply Power Law transformation
    img_gamma = np.power(img_float, gamma)
    # Convert image back to uint8 type
    img_gamma = (img_gamma * 255.0).astype(np.uint8)
    # Display the image
    cv2.imshow(f'Gamma: {gamma}', img_gamma)
    cv2.waitKey(0)
# Close all windows
cv2.destroyAllWindows()
```

The results (Screenshot)


Gamma: 0.2
Gamma: 0.5
Gamma: 1.2

Conclusion

In conclusion, Gray Level Transformations improve image processing by adjusting brightness, contrast, and detail visibility in grayscale images. Techniques like linear, logarithmic, and power-law transformations enhance pixel intensities, improving visual interpretation. This computationally efficient and adaptable method is crucial in fields like medical imaging and remote sensing for high-quality image analysis.